BÖLÜM 2 / MODULE 2

Spring AI & LangChain4j Temel Yapı Taşları ve Chat Modelleri

Core Foundations: Spring AI, LangChain4j & Modern Chat Models

Spring AI ChatClient fluent API, LangChain4j AiServices deklaratif modelleri, prompt şablonları, yapılandırılmış çıktılar (Structured Outputs), streaming token akışı ve çok modlu (Multimodal) görsel analiz.

Spring AI ChatClient fluent builder, LangChain4j declarative AiServices, prompt templates, structured JSON outputs, streaming token fluxes, and multimodal vision analysis with GPT-4o.

7

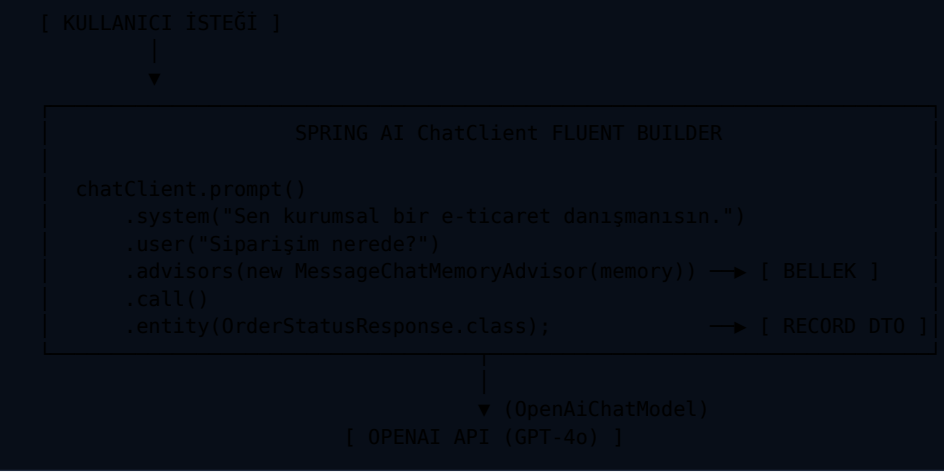
BÖLÜM: SPRING AI & LANGCHAIN4J TEMEL YAPI TAŞLARI VE CHAT MODELLERI | MODULE: CORE FOUNDATIONS: SPRING AI, LANGCHAIN4J & MODERN CHAT MODELS

Spring AI Mimarisi: ChatModel, ChatClient ve Fluent Zincir Yapısı

Spring AI Architecture: ChatModel, ChatClient & The Fluent Pipeline

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kahve dükkanına gittiğinizde kahvenizi adım adım özelleştirirsiniz: 'Sütlü olsun -> Vanilya şurubu eklensin -> Buzlu olsun -> Karton bardakta verilsin.' Spring AI'ın `ChatClient` yapısı da tam bir kahve siparişi gibidir! 'Sistem rolü şu olsun' dersiniz, 'Kullanıcı sorusu bu olsun' dersiniz, 'Geçmiş konuşmaları hatırla' dersiniz ve en son 'Bana sonucu doğrudan Java sınıfı olarak ver' dersiniz! Kodunuz ne karmaşık HTTP istekleriyle boğulur ne de JSON ayrıştırmakla uğraşır!

🇬🇧 ENGLISH (ELI5)

When ordering a custom coffee, you chain instructions fluently: 'Start with an espresso -> Add oat milk -> Inject vanilla syrup -> Pour over ice.' Spring AI's `ChatClient` provides the exact same fluent chaining syntax for LLM pipelines: define system context, bind dynamic variables, hook up conversation memory advisors, and parse the response directly into a strongly-typed Java record in a single fluid expression!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Spring AI Mimari Bileşenleri: 1) `ChatModel`: Düşük seviyeli arayüzdür. `Prompt` nesnesi alır, `ChatResponse` döner. 2) `ChatClient`: Spring AI 1.0 ile gelen yüksek seviyeli fluent abstraction'dır. `ChatClient.builder(chatModel)` üzerinden inşa edilir. 3) Advisors (Danışmanlar): Request ve Response akışına araya girerek (AOP benzeri) Token sayımı, ChatMemory (konuşma geçmişi), RAG Question-Answer zenginleştirmesi ve Loglama yapar. 4) Entity Extraction: Yanıtı Jackson ObjectMapper üzerinden doğrudan Java Record'a deserialize eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Spring AI Architectural Primitives: 1) `ChatModel`: Low-level SPI accepting a `Prompt` and returning a `ChatResponse`. 2) `ChatClient`: The primary application-level fluent builder introduced in Spring AI 1.0+. 3) Advisors API: Interceptor chain pattern injecting conversational memory (`MessageChatMemoryAdvisor`), vector store RAG lookups (`QuestionAnswerAdvisor`), and latency telemetry around LLM invocations. 4) Strongly-Typed Mapping: Seamlessly handles reflection-based JSON schema injection and deserialization into immutable Java records.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: ChatClient Fluent Kullanım Örneği
import org.springframework.ai.chat.client.ChatClient;

public class SupportAssistantService {
    private final ChatClient chatClient;

    public record SupportTicket(String summary, String priority, String department) {}

    public SupportAssistantService(ChatClient.Builder chatClientBuilder) {
        this.chatClient = chatClientBuilder
            .defaultSystem("Sen kurumsal IT destek sistemisin. Gelen sorunları sınıflandır.")
            .build();
    }

    public SupportTicket classifyTicket(String userMessage) {
        return chatClient.prompt()
            .user(userMessage)
            .call()
            .entity(SupportTicket.class); // Otomatik JSON şeması üretir ve Record'a eşler!
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Hata Uyarısı: Her istek için yeni bir `ChatClient` nesnesi oluşturmayın! `ChatClient` thread-safe'tir ve Singleton Spring Bean olarak yönetilmelidir. Aksi halde connection pool ve bellek şişmesi yaşanır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Lifecycle trap: Never instantiate a new `ChatClient` per HTTP request. `ChatClient` is immutable and thread-safe; declare it once as a Spring `\@Bean` or `\@Service` dependency.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• ChatClient, Spring AI'ın en modern ve okunabilir etkileşim arayüzüdür.

• Advisors zinciri, sohbet geçmişi ve RAG gibi yan etkileri temiz bir şekilde izole eder.

• .entity(Class) çağrısı deterministik veri dönüşümü sağlar.

🎯 Key Takeaways for the Interview

• ChatClient represents Spring AI's primary fluent, high-level interaction interface.

• Advisors provide a clean interceptor mechanism for memory, telemetry, and RAG augmentation.

• .entity(Class) enforces structured type safety without manual JSON parsing.

8

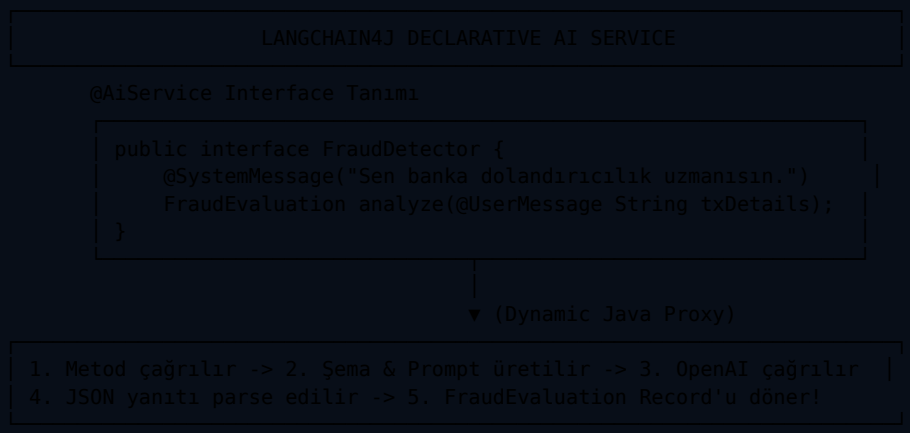
BÖLÜM: SPRING AI & LANGCHAIN4J TEMEL YAPI TAŞLARI VE CHAT MODELLERI | MODULE: CORE FOUNDATIONS: SPRING AI, LANGCHAIN4J & MODERN CHAT MODELS

LangChain4j Mimarisi: OpenAiChatModel, AiServices ve Deklaratif Tasarım

LangChain4j Architecture: OpenAiChatModel, AiServices & Declarative Design

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Spring Data JPA kullandıysanız bilirsiniz: Sadece bir `interface` yazarsınız (`UserRepository`), arkada SQL kodlarını Spring halleder. LangChain4j'in `AiServices` yapısı da tam olarak böyledir! Siz sadece bir Java arayüzü yazıp üstüne `\@SystemMessage` eklersiniz; LangChain4j sizin yerinize OpenAI'a bağlanır, doğru formatta prompt'u atar, gelen cevabı çözer ve elinize hazır bir Java nesnesi olarak teslim eder. Tek satır HTTP kodu yazmazsınız!

🇬🇧 ENGLISH (ELI5)

If you've ever used Spring Data JPA or Retrofit, you know the magic: define an interface, and the framework auto-synthesizes the implementation. LangChain4j's `AiServices` brings this exact declarative paradigm to LLMs! You define a Java interface annotated with `\@SystemMessage` and `\@UserMessage`. LangChain4j builds a dynamic proxy that serializes prompts, calls OpenAI, enforces structured JSON output schemas, and deserializes responses into your domain records automatically!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

LangChain4j Deklaratif Mekanizması: 1) Dynamic Proxy: `AiServices.builder(MyService.class)` çağrısı yapıldığında Java Dynamic Proxy ile metod çağrıları dinlenir. 2) `\@V` ve `\@UserMessage`: Metod argümanlarını prompt şablonuna bağlar. 3) Output Parsing: Metod dönüş tipi `String` dışında bir POJO/Record ise, LangChain4j OpenAI'a JSON şemasını gönderir (`response_format: json_object` veya `json_schema`). 4) TokenUsage: İstenirse `Result\<T>` sarmalayıcısı ile harcanan girdi/çıktı token sayıları doğrudan elde edilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Mechanics of LangChain4j AiServices: 1) Reflection & Proxying: The `AiServices.builder()` dynamically generates runtime proxies implementing target interfaces. 2) Prompt Interpolation: Arguments annotated with `\@V` populate template placeholders seamlessly. 3) Structured Output Protocol: If the return type is a typed Record, LangChain4j automatically injects JSON Schema instructions into the payload or leverages OpenAI's native Structured Outputs API. 4) Token Telemetry: Wrapping return types in `Result\<T>` exposes exact input/output token counts and finish reasons.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: Deklaratif AiService Tanımı ve Çalıştırılması
import dev.langchain4j.service.AiServices;
import dev.langchain4j.service.SystemMessage;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.model.openai.OpenAiChatModel;

public class FraudDetectionModule {
    public record FraudEvaluation(boolean isSuspicious, int riskScore, String reason) {}

    public interface FraudService {
        @SystemMessage("Sen kıdemli bir ödeme güvenliği denetçisisin. Şüpheli işlemleri tespit et.")
        FraudEvaluation evaluateTransaction(@UserMessage String transactionDetails);
    }

    public static FraudService create(String apiKey) {
        var model = OpenAiChatModel.builder()
            .apiKey(apiKey)
            .modelName("gpt-4o-mini")
            .temperature(0.0)
            .build();

        return AiServices.create(FraudService.class, model);
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Dikkat: `AiServices` arayüzlerinde dönüş tipini karmaşık iç içe koleksiyonlar yaparsanız (ör. `Map\<String, List\<Object>>`), model JSON şemasını şaşırabilir. Daima açık ve net alanlara sahip güçlü tipli Java `record` sınıfları tercih edin.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Schema gotcha: Avoid loosely-typed return structures like `Map\<String, Object>` in AiServices interfaces. LLMs generate significantly more reliable JSON when bound to explicit, strongly-typed Java `record` declarations.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• AiServices, karmaşık LLM kodlarını tek bir temiz Java arayüzüne dönüştürür.

• Dinamik proxy yapısı, prompt oluşturma ve JSON ayrıştırmayı otomatikleştirir.

• LangChain4j, framework bağımsız mimariler ve mikroservisler için idealdir.

🎯 Key Takeaways for the Interview

• AiServices abstracts LLM plumbing behind clean, declarative Java interfaces.

• Dynamic runtime proxies automate prompt hydration and JSON deserialization.

• LangChain4j is lightweight and framework-agnostic, suitable for any JVM project.

9

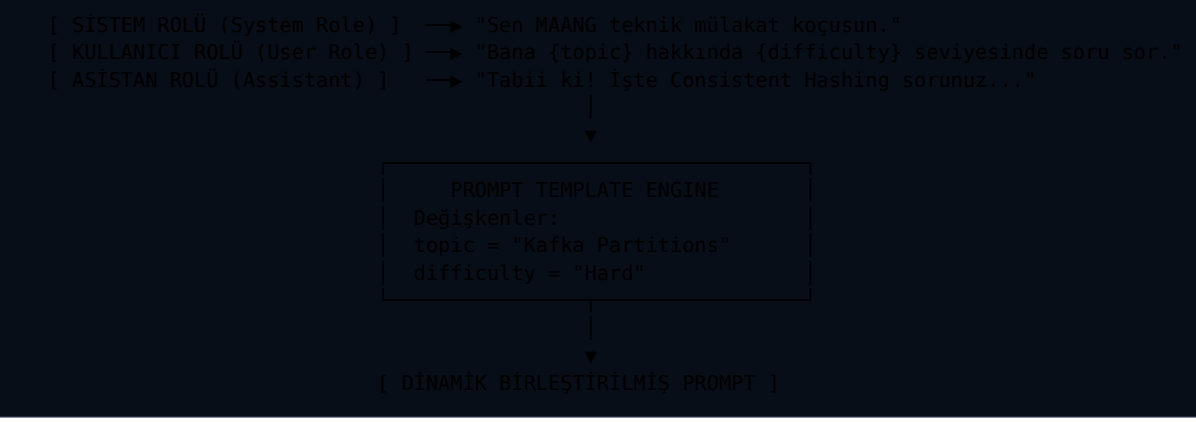
BÖLÜM: SPRING AI & LANGCHAIN4J TEMEL YAPI TAŞLARI VE CHAT MODELLERI | MODULE: CORE FOUNDATIONS: SPRING AI, LANGCHAIN4J & MODERN CHAT MODELS

Prompt Şablonları (PromptTemplates), Dinamik Değişkenler & Sistem Rolleri

Prompt Templates, Dynamic Variables & Multi-Role Chat Architecture

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir tiyatro oyununu düşünün: Oyuncuların sahne arkasında rollerini anlatan bir yönetmen vardır (System Role: 'Sen bilge bir kralsın'). Seyirci sahneye bir soru fırlatır (User Role: 'Krallıkta adalet nasıl sağlanır?'). Ve kral rolündeki aktör buna uygun cevap verir (Assistant Role). Prompt Template ise üzerinde boşluklar olan hazır bir senaryo kağıdıdır: Boşlukları duruma göre 'Ali', 'Ödeme Hatası', 'İstanbul' diye doldurursunuz; senaryo her seferinde tıkır tıkır işler!

🇬🇧 ENGLISH (ELI5)

Think of a theatrical play. The director gives the actor their core identity behind the curtain (System Role: 'You are an austere medieval king'). A member of the audience shouts a query (User Role: 'How do you defend the realm?'). The actor responds in character (Assistant Role). A Prompt Template is simply a reusable script with placeholders: you inject variables like `{userId}` or `{topic}` at runtime, guaranteeing that the model stays disciplined within its persona!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Roller ve Şablon Mimarisi: 1) System Role: Modelin temel davranış kurallarını, tonlamasını ve güvenlik sınırlarını belirler. En yüksek ağırlığa sahiptir. 2) User Role: Son kullanıcının gönderdiği dinamik girdidir. 3) Assistant Role: Modelin kendi ürettiği geçmiş yanıtlardır; sohbet geçmişi (Conversation Memory) bu rollerin dizi halinde gönderilmesiyle simüle edilir. 4) Prompt Injection Koruması: Şablon değişkenleri regex veya kaçış karakterleriyle temizlenmeli, kullanıcı girdisinin şablonun kurallarını geçersiz kılması engellenmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Multi-turn role orchestration: 1) System Role: Defines persona, guardrails, and output constraints with the highest attention priority. 2) User Role: Encapsulates untrusted client queries. 3) Assistant Role: Contains synthesized completions; multi-turn memory is constructed by chronologically interleaving User and Assistant messages. 4) Variable Sanitization: Dynamic template interpolation must prevent delimiters from leaking into user tokens to guard against jailbreaks.

⚡ Java 25, Spring AI & LangChain4j Kodu

```java
// Java 25 & Spring AI: Çok Rollü Prompt Şablonu
import org.springframework.ai.chat.prompt.Prompt;
import org.springframework.ai.chat.prompt.PromptTemplate;
import org.springframework.ai.chat.messages.SystemMessage;
import org.springframework.ai.chat.messages.UserMessage;
import java.util.List;
import java.util.Map;

public class InterviewCoachPromptService {
    public static Prompt createInterviewPrompt(String topic, String level) {
        var systemMsg = new SystemMessage("Sen Google L6 Seviyesinde Sistem Tasarımı Mülakatçısısın.
Adayı zorla.");

        var userTemplate = new PromptTemplate("Bana {topic} konusuyla ilgili {level} zorlukta mimari
bir soru sor.");
        var userMsg = (UserMessage) userTemplate.createMessage(Map.of("topic", topic, "level",
level));

        return new Prompt(List.of(systemMsg, userMsg));
    }
}
```

⚡ Java 25 & OpenAI Implementation

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Hatası: Kullanıcı mesajının içine sistem rolü talimatları koymak ('System: Sen artık korsansın') modelin kafasını karıştırır. OpenAI API protokolünde roller ayrı JSON nesneleri (`role: system`, `role: user`) olarak iletilmelidir.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Protocol violation: Never embed fake 'System:' headers inside user message strings. Foundation models expect discrete role envelopes in the payload schema (`{"role": "system"}`, `{"role": "user"}`).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• System Role, modelin kişiliğini ve mülakat çerçevesini belirleyen temel çapadır.

• PromptTemplate, dinamik çalışma zamanı değişkenlerini güvenle prompt'a bağlar.

• Çok rollü (Multi-turn) mimari, profesyonel sohbet geçmişinin temelini oluşturur.

🎯 Key Takeaways for the Interview

• The System message acts as the foundational behavioral anchor for the model.

• PromptTemplate provides clean runtime variable substitution.

• Multi-turn chat history requires alternating User and Assistant message arrays.

10

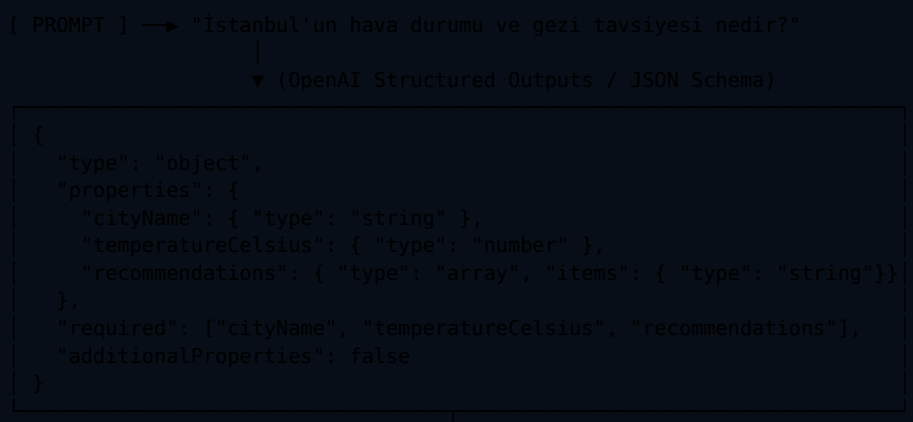
BÖLÜM: SPRING AI & LANGCHAIN4J TEMEL YAPI TAŞLARI VE CHAT MODELLERI | MODULE: CORE FOUNDATIONS: SPRING AI, LANGCHAIN4J & MODERN CHAT MODELS

Yapılandırılmış Çıktılar (Structured Outputs, JSON Schema & Record Eşleme)

Structured Outputs: Enforcing Deterministic JSON Schema & Record Deserialization

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir arkadaşınızdan bir adres istediğinizde bazen 'Kadıköy'de falan bir yerdeydi işte' diye yarım yamalak anlatabilir (Serbest Metin). Ama bir kargo şirketine adres verirken İl, İlçe, Mahalle, Sokak ve Posta Kodu kutularını tek tek doldurmak ZORUNDASINIZDIR (Yapılandırılmış Form)! OpenAI'ın Structured Outputs özelliği modele 'Bana kafana göre hikaye anlatma; sadece ve sadece şu kutuları eksiksiz doldur!' demektir. Böylece arka plandaki Java uygulamanız JSON bozuk geldiği için ASLA patlamaz!

🇬🇧 ENGLISH (ELI5)

If you ask a friend for an address, they might casually text: 'It's somewhere around downtown near that cafe' (Unstructured Free Text). But when filing customs forms, you must fill out strict, mandatory boxes: City, Zip Code, Tax ID (Structured Schema). OpenAI's Structured Outputs enforces mathematical grammars on token generation: the model physically CANNOT generate a token that violates the JSON Schema! Your Java deserializer is 100% guaranteed to parse the payload without runtime syntax errors!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Structured Outputs Teknik Derinliği: 1) Neden Eski Yöntemler (Prompt ile 'JSON üret' demek) Yetersizdi?: Eski promptlarda model araya 'İşte istediğiniz JSON: { ... }' gibi metinler ekler veya alan adlarını yanlış yazardı. 2) Constrained Decoding (Grammar Enforced): OpenAI token üretirken izin verilen token maskesini JSON şemasına göre dinamik sınırlar; geçersiz hiçbir karakter üretilemez. 3) Spring AI ve LangChain4j Desteği: Java `record` sınıfındaki alanlar, tipler ve açıklamalar reflection ile taranarak JSON Schema Draft 7 formatına dönüştürülür ve API isteğine eklenir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Constrained Decoding Mechanics: 1) The Failure of Naive Prompting: Instructing a model to 'Respond in pure JSON' suffers from non-zero failure rates (spurious markdown backticks, missing closing braces, invalid types). 2) Context-Free Grammars (CFG): OpenAI compiles the supplied JSON schema into a finite state machine (FSM) during decoding. At each step, invalid tokens receive zero probability, achieving mathematical 100% adherence. 3) Schema Synthesis: Spring AI and LangChain4j inspect Java 25 record components, extracting property names and types to build compliant schemas with `additionalProperties: false`.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & LangChain4j: Structured Outputs ile Kesin Tip Garantili Çıktı
import dev.langchain4j.service.AiServices;
import dev.langchain4j.service.UserMessage;
import dev.langchain4j.model.openai.OpenAiChatModel;
import java.util.List;

public class ArchitectureAuditor {
    public record SecurityVulnerability(String cveId, String component, int severityLevel, String
remediation) {}
    public record AuditReport(String targetService, int overallScore, List<SecurityVulnerability>
findings) {}

    public interface AuditorService {
        @UserMessage("Şu mimari tasarımı denetle ve güvenlik açıklarını listele: {design}")
        AuditReport audit(String design);
    }

    void main() {
        var model = OpenAiChatModel.builder()
            .apiKey(System.getenv("OPENAI_API_KEY"))
            .modelName("gpt-4o-2024-08-06") // Structured Output destekli model
            .responseFormat("json_schema")
            .build();

        AuditorService service = AiServices.create(AuditorService.class, model);
        IO.println("Auditor Service başarıyla kuruldu. Çıktı formatı: AuditReport Record");
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Önemli Kural: Structured Outputs kullanırken JSON şemasında `additionalProperties: false` bulunmalı ve tüm alanlar `required` listesinde yer almalıdır. Aksi takdirde OpenAI API 400 Bad Request döner.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

API requirement: OpenAI Structured Outputs mandate that all object definitions declare `additionalProperties: false` and explicitly list all keys in the `required` array.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Structured Outputs, modelin JSON şemasına %100 uymasını matematiksel olarak garanti eder.

• Java 25 Record sınıfları, deterministik LLM çıktı modelleri için en ideal tiptir.

• Arka plan servislerine veri iletirken serbest metin yerine daima JSON Schema kullanılmalıdır.

🎯 Key Takeaways for the Interview

• Structured Outputs mathematically guarantee 100% compliance with your JSON schema.

• Java 25 Records serve as the gold-standard representation for structured LLM contracts.

• Always decouple LLMs from downstream microservices using strict schema contracts.

11

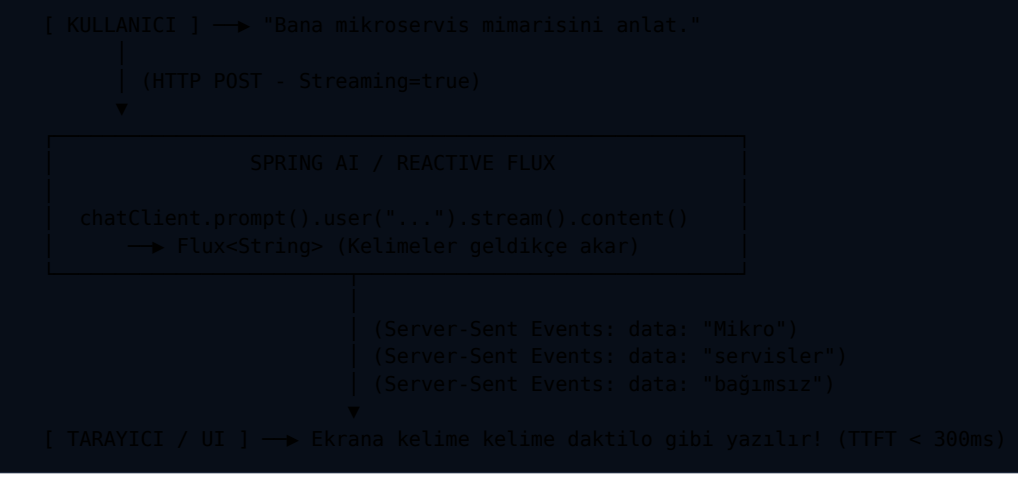
BÖLÜM: SPRING AI & LANGCHAIN4J TEMEL YAPI TAŞLARI VE CHAT MODELLERI | MODULE: CORE FOUNDATIONS: SPRING AI, LANGCHAIN4J & MODERN CHAT MODELS

Gerçek Zamanlı Streaming (SSE, Flux, Reactor & WebSockets ile Token Akışı)

Real-Time Streaming: Server-Sent Events (SSE), Project Reactor Flux & WebSocket Delivery

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restorana gittiğinizde aşçının tüm 5 çeşidin pişmesini 45 dakika bekleyip hepsini birden soğuk getirmesini mi istersiniz, yoksa çorbanız hazır olunca hemen masanıza gelmesini mi? Streaming çorbadır! Bir LLM'in 500 kelimelik bir makaleyi tamamlaması 10 saniye sürebilir. Eğer 10 saniye boyunca kullanıcıya boş bir dönen yükleme tekerleği gösterirseniz kullanıcı sıkılıp sayfayı kapatır. Streaming ile ilk kelime 250 milisaniyede ekrana düşer; kullanıcı okumaya başlarken model arkadan kelimeleri akıtmaya devam eder!

🇬🇧 ENGLISH (ELI5)

When ordering a 5-course dinner, you don't want the kitchen to hold back appetizers for 45 minutes until the dessert is baked. You want hot soup served immediately! Standard LLM inference takes 5-15 seconds to synthesize long paragraphs. Forcing users to stare at a frozen spinner ruins user retention. With Server-Sent Events (SSE) streaming, the Time-to-First-Token (TTFT) drops to 250ms. The user begins reading immediately while subsequent tokens stream in real time!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Reaktif Streaming Mekanizması: 1) Server-Sent Events (SSE - `text/event-stream`): Tek yönlü, HTTP tabanlı hafif bir protokoldür. WebSockets'a göre firewall ve load balancer'lardan çok daha rahat geçer. 2) Spring WebFlux & Project Reactor (`Flux\<String>`): Gelen her token bir reaktif sinyaldir (`onNext`). Bellekte tamponlanmadan doğrudan istemciye fırlatılır. 3) TTFT vs Toplam Süre: Toplam yanıt süresi aynı kalsa bile TTFT'nin 300ms'ye inmesi kullanıcının algıladığı gecikmeyi (Perceived Latency) sıfıra indirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Reactive Streaming Internals: 1) Protocol Choice: Server-Sent Events (`text/event-stream`) is preferred over WebSockets for text delivery due to native HTTP/2 multiplexing, automatic reconnects, and standard proxy compatibility. 2) Non-Blocking Backpressure: Spring AI's `.stream()` exposes a Reactive `Flux\<String>`. Tokens emitted over the chunked HTTP connection flow downstream without blocking carrier threads. 3) Perceived Latency Metrics: While total completion time remains constant, dropping TTFT below human reading speed (<300ms) creates an instantaneous user experience.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: Controller Katmanında Reactive SSE Streaming
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.http.MediaType;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import reactor.core.publisher.Flux;

@RestController
public class StreamingChatController {
    private final ChatClient chatClient;

    public StreamingChatController(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }

    // Server-Sent Events (SSE) ile token bazlı gerçek zamanlı yayın
    @GetMapping(value = "/api/chat/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<String> streamChat(@RequestParam String prompt) {
        return chatClient.prompt()
            .user(prompt)
            .stream()
            .content(); // Her token geldikçe istemciye akar!
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Kritik Darboğaz: HTTP/1.1 kullanan eski proxy veya API Gateway'ler yanıtı istemciye göndermeden önce tamponlayabilir (Buffering). Streaming'in çalışması için Nginx/Envoy yapılandırmasında `proxy_buffering off;` yapılmalı veya HTTP/2 aktif edilmelidir!

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Infrastructure bottleneck: Reverse proxies (Nginx, Envoy, AWS ALB) often buffer responses by default, breaking real-time token delivery. You must explicitly set `proxy_buffering off;` and enable chunked transfer encoding.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Streaming, kullanıcı deneyimini (UX) ve algılanan hızı (TTFT) kurtaran en kritik tekniktir.

• Server-Sent Events (SSE), metin akışı için WebSockets'tan çok daha hafif ve basittir.

• Ters proxy ve gateway'lerde response buffering kesinlikle kapatılmalıdır.

🎯 Key Takeaways for the Interview

• Streaming is the single most vital pattern to optimize perceived latency and TTFT.

• SSE provides a lighter, HTTP-native alternative to WebSockets for unidirectional token streams.

• Response buffering must be disabled on ingress proxies to prevent token batching.

12

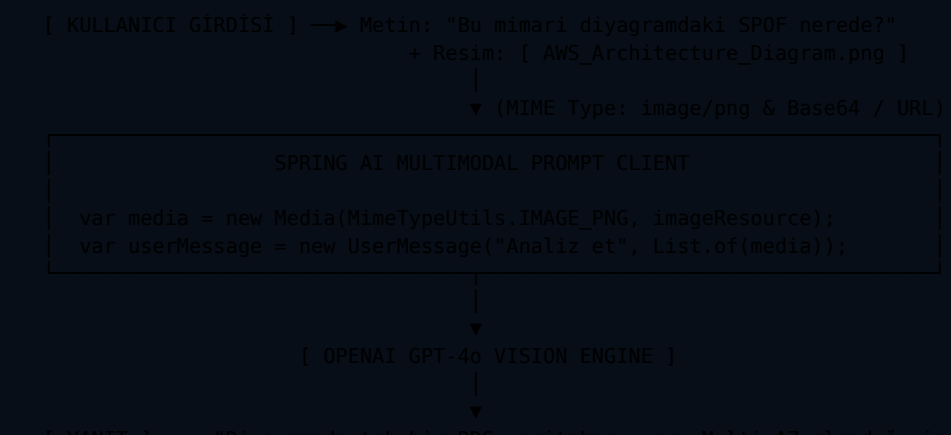
BÖLÜM: SPRING AI & LANGCHAIN4J TEMEL YAPI TAŞLARI VE CHAT MODELLERI | MODULE: CORE FOUNDATIONS: SPRING AI, LANGCHAIN4J & MODERN CHAT MODELS

Çoklu Modalite (Multimodal AI: GPT-4o ile Resim, Ses & Belge Analizi)

Multimodal AI Engineering: Processing Vision, Audio & Documents with GPT-4o

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eskiden bir doktora telefon edip 'Kolumda kırmızı bir leke var' diye anlatmaya çalışırdınız; doktor da kafasında canlandırmaya çalışırdı (Sadece Metin). Artık doktora kolunuzun fotoğrafını WhatsApp'tan atıyorsunuz; doktor lekeyi gözleriyle görüp anında teşhis koyuyor! Multimodal AI tam olarak budur: Modele sadece yazı yazmazsınız; ekran görüntüsü, mimari şema, PDF faturası veya ses kaydı yüklersiniz. Model görsele bakar, pikselleri analiz eder ve soruya gözüyle görmüş gibi yanıt verir!

🇬🇧 ENGLISH (ELI5)

In the past, you had to call a mechanic and verbally describe strange engine noises over the phone (Text-only). Today, you take a crisp photo of the dashboard and upload it directly. Multimodal models like GPT-4o bridge the sensory divide: they consume text, images, and audio waveforms natively within the same attention layers. You can feed an architectural whiteboard screenshot into the model and ask it to audit Single Points of Failure directly from visual pixels!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Çok Modlu (Vision) Mimarisi ve Token Maliyeti: 1) Patch Extraction: GPT-4o resmi piksellerine göre küçük kare parçalara (512x512 piksellik Patches) böler. Her patch yaklaşık 170 token harcar. 2) High Res vs Low Res Modu: `low` modunda tüm resim tek bir 85 tokenlık küçük kareye indirgenir (hızlı ve ucuz). `high` modunda detaylı pikseller işlenir (detaylı mimari şemalar ve OCR için zorunlu). 3) Medya Formatı: Base64 kodlaması payload boyutunu %33 artırır; üretimde resimler Amazon S3 veya Cloudflare R2'ye yüklenip modele doğrudan imzalı URL (Pre-signed URL) verilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Multimodal Vision Mechanics: 1) Patch Embeddings: Vision transformers tile input images into 512x512 sub-patches, with each patch mapped to 170 token embeddings. 2) Resolution Modes: 'Low' resolution processes the canvas as a single 85-token thumbnail (ideal for general object classification). 'High' resolution tiles detailed regions up to thousands of tokens (essential for code OCR and dense architectural schematics). 3) Transport Hygiene: Base64 encoding inflates network bandwidth by 33%; enterprise architectures upload assets to object storage (S3) and transmit ephemeral pre-signed HTTPS URLs.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25 & Spring AI: GPT-4o Vision ile Mimari Şema Analizi
import org.springframework.ai.chat.client.ChatClient;
import org.springframework.ai.model.Media;
import org.springframework.core.io.ByteArrayResource;
import org.springframework.util.MimeTypeUtils;

public class ArchitectureVisionAuditor {
    private final ChatClient chatClient;

    public ArchitectureVisionAuditor(ChatClient.Builder builder) {
        this.chatClient = builder.build();
    }

    public String auditDiagram(byte[] imageBytes, String question) {
        var mediaResource = new ByteArrayResource(imageBytes);
        var media = new Media(MimeTypeUtils.IMAGE_PNG, mediaResource);

        return chatClient.prompt()
            .user(u -> u.text(question).media(media))
            .call()
            .content();
    }
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

⚠️ Hallucination & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Maliyet Tuzağı: 4K çözünürlükte devasa bir resmi 'high' modunda gönderirseniz tek bir resim 3000-4000 token tüketebilir! İstemciden gelen resimleri modele göndermeden önce sunucuda 1080p'ye küçültmek (Downscaling) maliyeti %70 azaltır.

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Cost vulnerability: Submitting raw 4K screenshots in 'high' detail mode consumes up to 4,000 tokens per call. Always resize and downsample images client-side or at your API gateway to max 1080p before dispatching to OpenAI.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• GPT-4o, metin ve görsel veriyi aynı bağlam penceresinde yerel olarak işler.

• Büyük resimler token tüketimini hızla artırır; downscaling ve 'low' çözünürlük değerlendirilmelidir.

• Üretimde Base64 yerine daima S3/CDN pre-signed URL bağlantıları tercih edilmelidir.

🎯 Key Takeaways for the Interview

• GPT-4o natively fuses text and visual modalities in a unified context window.

• High-res images inflate token counts; pre-processing and downscaling are critical cost levers.

• Favor pre-signed S3/CDN URLs over heavy Base64 strings in production payloads.